



# Indian Institute of Technology, Roorkee

## Technical Report

### Campus Ride Platform

#### A Real-Time Campus Mobility and Ride Platform

---

#### CONTACT INFO:

1. **Name** : Lakshya Gupta

2. **Contact Number** : 9837348350

3. **Branch**: Geological Technology

4. **Email ID**: [lakshya\\_g@es.iitr.ac.in](mailto:lakshya_g@es.iitr.ac.in)

# Problem Understanding

Large campuses such as IIT Roorkee rely heavily on e-rickshaws for last-mile transport, but ride requests and driver availability are coordinated through informal, fragmented channels. This causes long waits, uneven demand distribution, idle drivers, and a poor experience for everyone.

The goal is a **centralized, real-time platform** that connects passengers and drivers within the campus. Passengers should discover available drivers, request rides, and track them live; drivers should see incoming requests, accept them, and manage their trips. The system must keep ride state, assignments, and driver availability **consistent across all users in real time**.

## Core requirements addressed:

- Secure authentication and profile management for passengers and drivers (with email OTP verification).
- Driver availability management (online/offline + live location).
- Ride request -> assignment -> lifecycle workflow, ensuring a ride is assigned to **exactly one driver**.
- Real-time updates over WebSockets (the core engineering challenge).
- Driver dashboard with stats, history, and ratings.
- Ratings & feedback influencing driver reputation.

ALSO ADDED : live maps, ride scheduling, simulated digital payments (UPI/Card/Wallet/Cash), demand analytics, and ML-based demand forecasting.

**Design priorities:** reliability of the core ride workflow, low-latency real-time updates, data consistency, and a clean, scalable, maintainable codebase and best thing is all built with a **fully free** technology stack (no paid APIs).

---

---

# System Architecture

The platform is a single full-stack application. A custom Node server (server.js) runs the **Next.js app** (UI + REST API) and the **Socket.IO server** in one process, so API handlers and the WebSocket layer share one runtime and one Socket.IO instance (exposed via globalThis).

## Layers:

- **Client (browser):** React UI (Next.js App Router) plus a Socket.IO client connected to the /rides namespace for live updates.
- **Server:** Next.js routing/SSR, REST API routes under /app/api/\*, JWT authentication with bcrypt, and the Socket.IO server (/rides, /drivers namespaces).
- **Database:** PostgreSQL (Supabase) accessed through the Prisma ORM.
- **Email:** SMTP (Gmail) for OTP delivery.

**Real-time principle - "persist first, broadcast second":** every state change is written to the database, and only *after* the commit does the API emit the corresponding Socket.IO event. This guarantees clients never receive an update that isn't already durable.

**Typical ride request flow:** Passenger calls POST /api/rides -> row persisted -> server emits ride:requested to all online drivers -> a driver calls PATCH /api/rides/:id {ACCEPTED} (atomic claim) -> server emits ride:accepted to the passenger's personal room -> both UIs update instantly with no refresh.

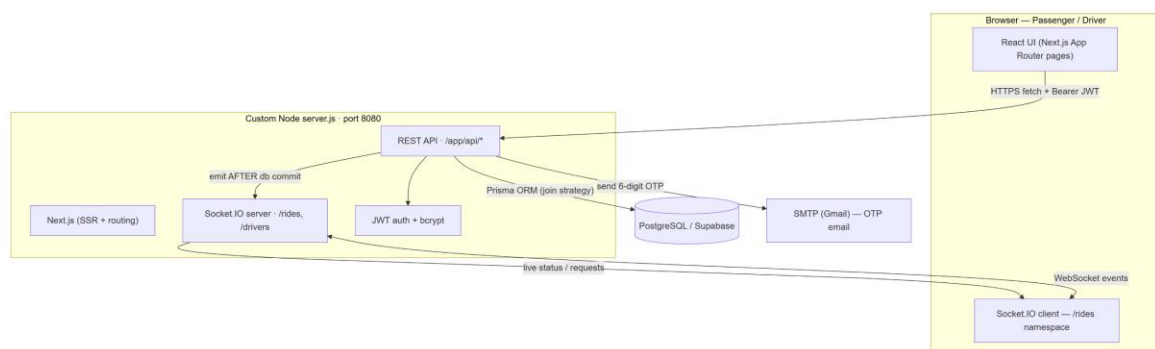


Figure 1: High-level system architecture and data flow.

## Ride Lifecycle (State Management)

A ride moves through a strict state machine. Each transition is persisted and then broadcast over Socket.IO, keeping all connected clients consistent.

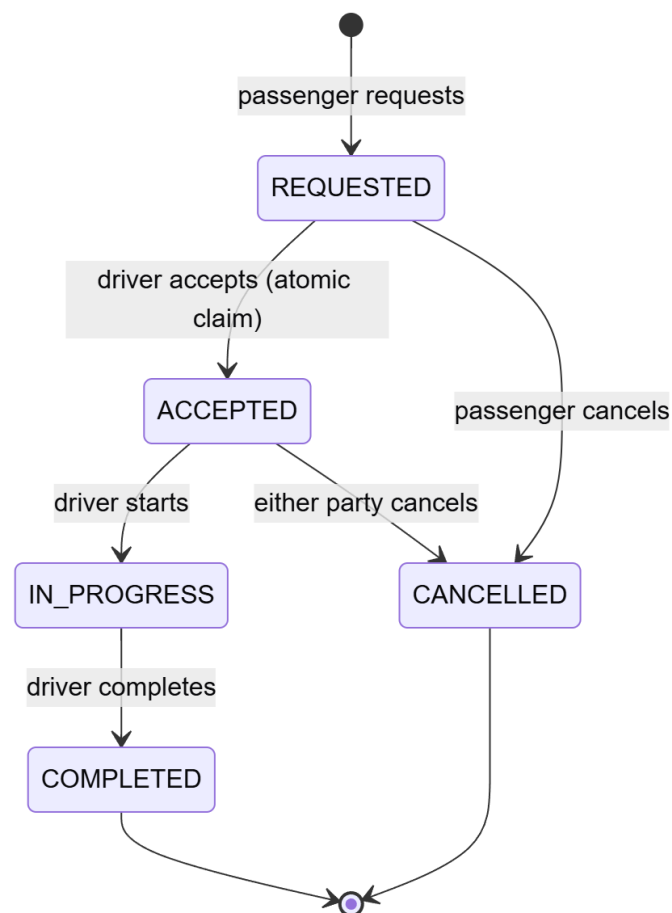


Figure 2: Ride state machine (*REQUESTED* -> *ACCEPTED* -> *IN\_PROGRESS* -> *COMPLETED*, with *CANCELLED* reachable from active states).

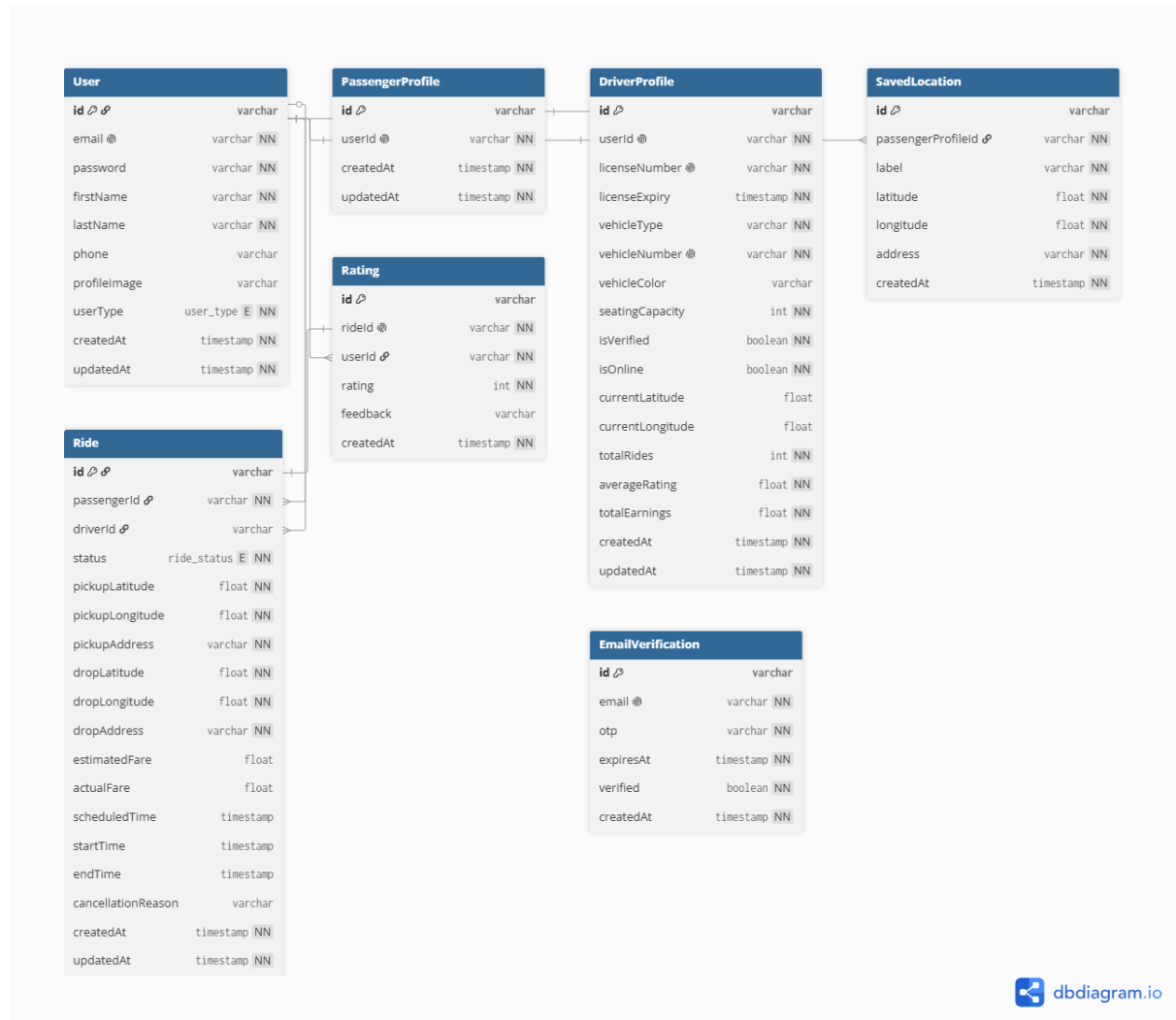
Assignment is made race-safe with a conditional database update: accepting a ride only succeeds while it is still *REQUESTED* and unassigned, so concurrent drivers cannot both win the second receives a 409 Conflict.

---

---

# Database Schema

PostgreSQL via Prisma ORM. **7 tables**, string cuid primary keys, with foreign keys, unique constraints, and indexes on frequently queried fields.



## Enumerations

- UserType: PASSENGER, DRIVER
- RideStatus: REQUESTED, ACCEPTED, IN\_PROGRESS, COMPLETED, CANCELLED

## Integrity & performance rules

- Unique constraints on User.email, DriverProfile.licenseNumber, DriverProfile.vehicleNumber, Rating.rideId, EmailVerification.email.
- Rating.rideId unique  $\Rightarrow$  each ride can be rated **once**.

- Cascade deletes: User -> its profiles; Ride \_> its rating.
- Indexes on Ride(status, passengerId, driverId, createdAt) to speed up open-request lookups, per-user history, and analytics time-window scans.

## Entity Relationship Diagram (ERD)

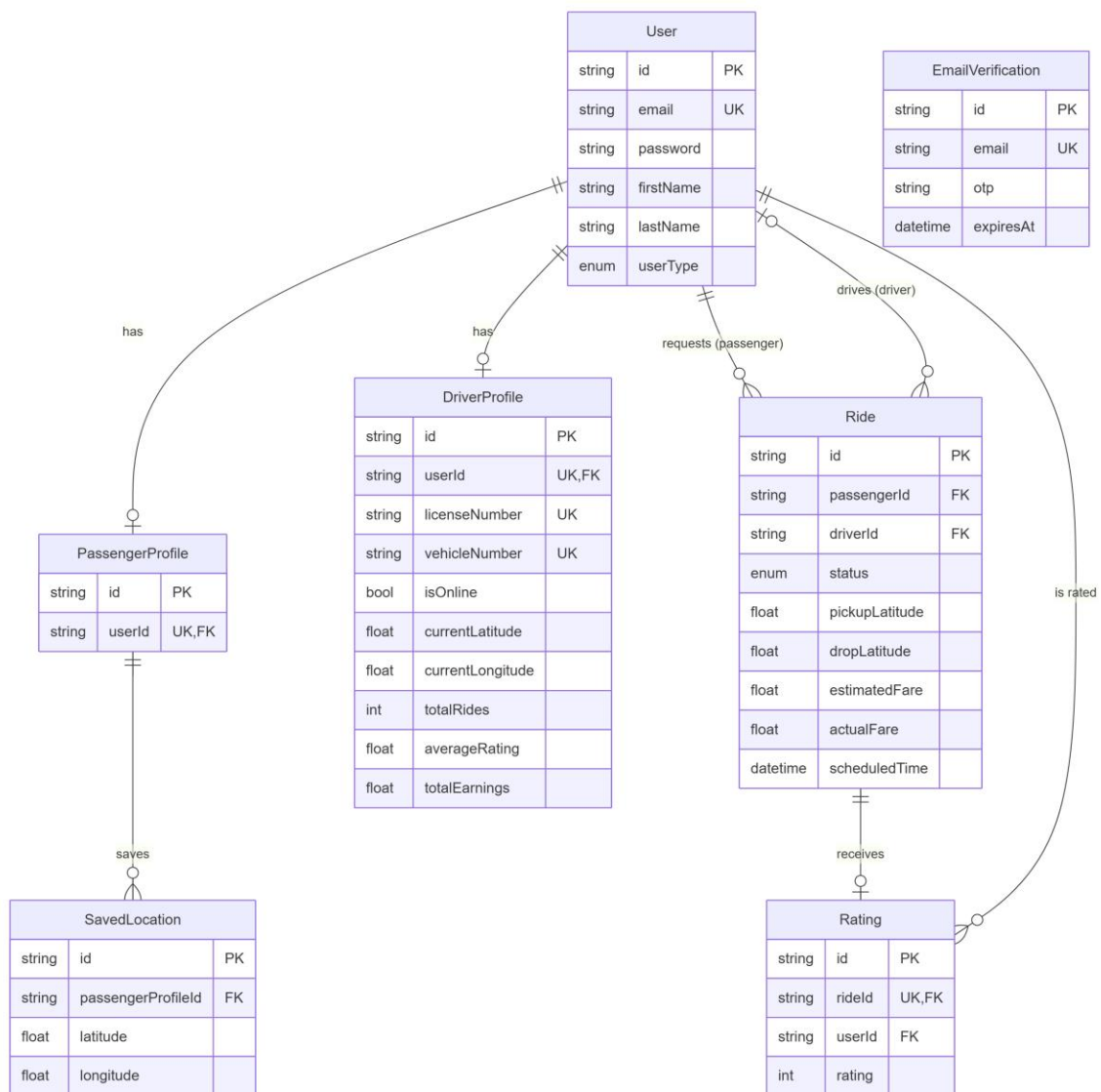


Figure 3: Entity Relationship Diagram.

### Relationships:

- **User - PassengerProfile** : one-to-one (a user has at most one passenger profile).
- **User - DriverProfile** : one-to-one (a user has at most one driver profile).

- **PassengerProfile - SavedLocation** : one-to-many.
- **User - Ride (as passenger)** : one-to-many (Ride.passengerId).
- **User - Ride (as driver)** : one-to-many, optional (Ride.driverId, null until accepted).
- **Ride - Rating** : one-to-one (Rating.rideId unique).
- **User - Rating** : one-to-many (a driver accumulates many ratings).

*EmailVerification is an intentionally standalone lookup table, keyed by email rather than linked by a foreign key, because it must exist before the User row is created during signup.*

---



---

## API Overview

All protected routes require an Authorization: Bearer <JWT> header. Request bodies are validated with Zod. Responses follow { message, ... } on success and { error } with an appropriate HTTP status on failure.

### REST Endpoints

| Method      | Endpoint                | Purpose                                               |
|-------------|-------------------------|-------------------------------------------------------|
| POST        | /api/auth/send-otp      | Generate + email a 6-digit OTP                        |
| POST        | /api/auth/signup        | Create account (requires valid OTP)                   |
| POST        | /api/auth/login         | Verify credentials, return JWT                        |
| GET / POST  | /api/rides              | List my rides / create a ride request (computes fare) |
| GET         | /api/rides/requests     | Open unassigned requests (drivers)                    |
| GET / PATCH | /api/rides/:id          | Ride detail / status change                           |
| POST        | /api/rides/:id/ratings  | Rate a completed ride                                 |
| GET / POST  | /api/rides/scheduled    | List / create scheduled rides                         |
| GET         | /api/drivers/available  | Online drivers near a point (Haversine)               |
| GET / PATCH | /api/drivers/status     | Driver availability + live location                   |
| POST        | /api/payments/initiate  | Simulated payment; persists <code>actualFare</code>   |
| GET         | /api/analytics/demand   | Hourly/daily demand, peak hours, top locations        |
| GET         | /api/analytics/forecast | 7-day ML demand forecast                              |

### Socket.IO events (/rides namespace)

| Event                       | Direction          | Meaning                                         |
|-----------------------------|--------------------|-------------------------------------------------|
| register                    | client → server    | Join personal room <code>user-&lt;id&gt;</code> |
| join-ride                   | client → server    | Follow ride room <code>ride-&lt;id&gt;</code>   |
| ride:requested              | server → drivers   | New open request broadcast                      |
| ride:accepted               | server → passenger | A driver claimed the ride                       |
| ride:status_updated         | server → ride room | Lifecycle change                                |
| driver:availability_changed | server (/drivers)  | Online driver pool changed                      |

## Design Decisions

1. **One process for HTTP + WebSocket.** A custom server.js runs Next.js and Socket.IO together; the IO instance is shared via globalThis so API routes can emit after committing to the database ("persist, then broadcast").
2. **Atomic ride assignment.** Accept uses a conditional updateMany(where: { id, status: REQUESTED, driverId: null }). The database guarantees exactly one driver wins; the loser receives 409 Conflict no application locks, no race conditions.
3. **Stateless JWT auth.** 7-day bearer tokens with bcrypt-hashed passwords; no server session store, enabling horizontal scaling.
4. **Email OTP verification.** Two-step signup; OTP stored in EmailVerification with a 10-minute expiry, delivered via SMTP, with a console fallback so the flow works with zero setup in development.
5. **Single-query relation loading.** Ride reads use Prisma's relationLoadStrategy: "join" to fetch relations in one SQL JOIN instead of multiple round-trips roughly halving latency to the remote database.
6. **Optimistic UI for real-time actions.** Driver accept/start/complete update local state instantly and reconcile in the background, so the interface feels instant despite network latency.
7. **Deterministic fare model.** ₹20 base + ₹12/km (minimum ₹25) from Haversine distance, shared by the client preview and the server, so estimates always match the charged amount.
8. **Preset campus locations.** A fixed list of IIT Roorkee landmarks gives every route real, distinct coordinates (and therefore distinct fares) without a paid geocoding API.



9. **Lightweight, transparent ML.** Forecasting combines Simple Moving Average and Exponential Smoothing over historical daily ride counts — dependency-free and explainable.
10. **Fully free stack.** Next.js, Socket.IO, Prisma/PostgreSQL, Leaflet/OpenStreetMap, Recharts, and simulated payments — no paid services anywhere.

**Technology stack:** Next.js 15 + TypeScript (strict), Socket.IO, PostgreSQL (Supabase) + Prisma, JWT + bcryptjs, Zod, Nodemailer, Tailwind CSS, Leaflet, Recharts.